

Rust's Stack, Ownership, and Scope Model

How Rust registers pointers on the stack, how the borrow checker governs frame lifetimes, and how to design code that flows with the compiler.

1. The Rust Stack vs The C Stack

C pointer on stack

```
void foo() {
    int x = 42;           // x lives on stack
    int* p = &x;          // p points INTO the stack frame
    *p = 99;              // fine - p is valid while foo() is alive
}
// x and p both dead here - frame popped
```

C lets you take a pointer to a stack variable and **pass it upward**:

```
int* dangling() {
    int x = 42;
    return &x; // COMPILES - but x is dead after return!
}             // caller gets a dangling pointer
```

Rust reference on stack

```
fn foo() {
    let x: i32 = 42;           // x on stack
    let p: &i32 = &x;          // p borrows x - borrow checker tracks this
    // *p == 42 - valid while x is alive
}
// p dead here, x dead here - both popped
```

Rust **prevents** the dangling case at compile time:

```
fn dangling() -> &i32 {
    let x = 42;
    &x // COMPILE ERROR: `x` does not live long enough
}
```

How the frame changes

Aspect	C	Rust
Stack pointer to local	Allowed (dangling risk)	Allowed (checked by borrows)
Return pointer to local	Compiles (UB)	Compile error
Frame layout	Fixed at compile time	Fixed at compile time
Drop of local	Manual or <code>goto cleanup</code>	Automatic at end of scope
Register spills	Compiler decides	Compiler decides (same backend LLVM)

Key insight: Rust doesn't change the *mechanical* stack frame — the CPU sees the same push/pop/sp. What changes is the **compiler's ability to prove** when a pointer is still valid. This lets it:

1. Insert `drop()` calls at the exact right point
2. Reject code that would create dangling references
3. Optimize away moves (no runtime cost)

2. The "Schedule" — Deterministic Drop

Rust's **drop schedule** is computed at compile time for every scope.

```
fn schedule_demo() {
    let a = String::from("hello"); // a allocated on heap, ptr on stack
    let b = String::from("world"); // b allocated
    // ... use a and b ...
    drop(a);                        // (1) explicit – frees heap, pops stack ptr
    // b still alive here
}                                  // (2) b dropped here implicitly

// Compiler-generated pseudo-code:
// fn schedule_demo() {
//     stack_push(a_ptr);           // "register" pointer on stack
//     stack_push(b_ptr);
//     // ...
//     drop_in_place(&mut a);       // free heap, invalidate stack slot
//     drop_in_place(&mut b);       // free heap, invalidate stack slot
//     stack_pop(16);               // pop frame (2 × 8 bytes for ptrs)
// }
```

The exact drop order rules

Variables drop in REVERSE declaration order

```
let a = ...;    ← allocated first
let b = ...;    ← allocated second
let c = ...;    ← allocated third

// use them...

drop(c);        ← c dropped first
drop(b);        ← b dropped second
drop(a);        ← a dropped last
```

Nested scopes create a nested drop schedule

```
fn scoped_schedule() {
    let a = String::from("outer");    // ┌ frame slot 1
    {                                  // |
        let b = String::from("inner"); // ┌ frame slot 2
        // both alive here             | |
    }                                  // | b dropped here (pop slot 2)
    // only a alive here                |
    // a used here...                   |
}                                       // └ a dropped here (pop slot 1)
```

This is **deterministic** — the same inputs always produce the same schedule. No GC pause, no reference counting overhead, no sweep phase.

3. The Scope Limiter — How the Borrow Checker Adjusts the Frame

The **borrow checker** is the "scope limiter." It adjusts the *logical* frame by:

3a. Shortening reference lifetimes

```
fn borrow_limits_frame() {
    let data = vec![1, 2, 3];    // data lives until end of function

    let r: &Vec<i32> = &data;    // r borrows data
```

```
// compiler tracks: r must die before data can be mutated or moved
println!("{}", r.len());           // r used here

// r is no longer used – borrow checker considers it "dead" here
// even though the variable `r` still exists in the frame!

let mut data_mut = data;           // OK – r is "dead", so move is fine
data_mut.push(4);

}
// data_mut dropped here
```

The borrow checker **tracks liveness at the statement level**, not the variable level. This means the **effective frame** is smaller than the lexical frame.

3b. Non-Lexical Lifetimes (NLL)

Before NLL (2018), Rust used only lexical scopes:

```
// PRE-NLL (would NOT compile):
let mut v = vec![1, 2, 3];
let r = &v[0];           // borrow starts
println!("{}", r);        // borrow used
v.push(4);                 // ERROR: can't mutably borrow while immutably borrowed
// borrow ends here lexically (end of scope)
```

With NLL:

```
// NLL (compiles since Rust 2018):
let mut v = vec![1, 2, 3];
let r = &v[0];           // borrow starts
println!("{}", r);        // borrow last used HERE ← NLL ends borrow here
v.push(4);                 // OK – borrow already considered dead
```

Effect on frame: The compiler knows `r`'s slot in the frame is reusable after the last use. This lets subsequent code reuse that stack slot for something else (via LLVM optimization).

3c. The borrow checker as a finite state machine

The compiler models each reference as a state machine:

```
Active —(last use)—> Dead
  |
  └─(error)—> Invalid    |
                        └─(drop occurs here)
```

Each scope is a **schedule machine**:

Schedule for a frame:

1. Push variables (in declaration order)
2. Track borrow start/end for each reference
3. At each statement: verify all borrows are consistent
4. At each last-use: mark reference as "dead", slot reusable
5. At } for scope: drop all alive variables in reverse order
6. Pop frame

4. How to Design Code That Flows With This Actuation

Principle 1: Put data in the smallest scope possible

```
// BAD – holds borrow long
fn bad(data: &[u8]) -> &[u8] {
    let mut temp = Vec::new();
    temp.extend_from_slice(data);

    let processed = &temp[..2]; // borrow starts
    processed           // borrow returned → extends to caller's scope
}

// GOOD – scope limits the borrow, clone what you need
fn good(data: &[u8]) -> Vec<u8> {
    let mut temp = Vec::new();
    temp.extend_from_slice(data);
    let result = temp[..2].to_vec(); // copy out, no borrow held
    result
}
```

Principle 2: Use blocks to limit borrows

[illegible]

```
// Stage 2: mutation
data.push(stats.0 / stats.1 as i32); // OK – borrow from stage 1 is dead
}
```

Principle 3: Prefer owned data at scope boundaries

```
// BAD – fights the borrow checker; tangled lifetimes
fn find_and_modify<'a>(items: &'a mut Vec<String>, key: &'a str) -> &'a mut String {
    let found = items.iter_mut().find(|s| s.contains(key)).unwrap();
    found
}

// GOOD – split into scoped operations
fn find_and_modify(items: &mut Vec<String>, key: &str) {
    let idx = items.iter().position(|s| s.contains(key));
    if let Some(i) = idx {
        items[i].push_str(" (modified)");
    }
}
```

Principle 4: Use the type system to encode scope constraints

```
// Scope is encoded in the type:
fn takes_ref<'a>(x: &'a i32) -> &'a i32 { x }
//
//      ┌───┐
//      │   │
//      │   └─ return lives as long as input
//      │
//      └─ input lives at least 'a

// Design your APIs so the types tell the borrow checker what you mean:
struct Database {
    conn: String,
}

struct Transaction<'a> {
    db: &'a Database,
}

impl<'a> Transaction<'a> {
    fn commit(&self) {
        println!("committing to {}", self.db.conn);
    }
} // Transaction dies here, releasing the borrow
```

```
fn scope_limited_api(db: &Database) {
    let tx = Transaction { db };    // borrow starts
    tx.commit();                    // last use of tx
    // tx is dead here – db is free again
    println!("{}", db.conn);       // OK
}
```

Principle 5: Leverage drop order for resource safety

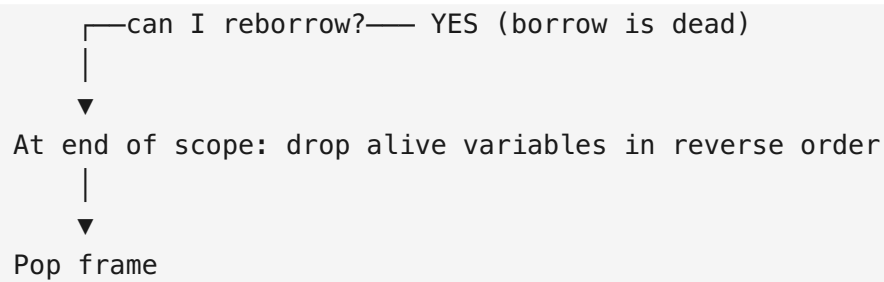
```
struct LogGuard;
impl Drop for LogGuard {
    fn drop(&mut self) {
        println!("[LOG] operation ended");
    }
}

fn monitored_operation() {
    let guard = LogGuard;           // ← pushed first
    let data = String::from("work"); // ← pushed second
    // ... do work ...

    // data dropped first (frees heap)
    // guard dropped second (logs ending)
    // ← even if data.drop() panics,
    //   guard still runs!
```

Flow chart: How to reason about your frame

```
Write a scope:
|
▼
List the variables (stack allocations):
|
▼
List the borrows (references):
|
▼
For each borrow, find its LAST USE:
|
▼
After last use → borrow is "dead" (even if variable is still in scope)
|
▼
Between last use and end of scope:
└─can I mutate?── YES (borrow is dead)
└─can I move?── YES (borrow is dead)
```



5. Visual Example: Frame Schedule in Action

```
fn frame_schedule_demo() {  
    // ┌ FRAME PUSH ───────────────────┐  
    // │ SP→ [   a   ] String (ptr+len+cap on stack)│  
    // │       [   b   ] String                │  
    // │       [ r_a ] &String (reference to a)   │  
    // │       [ r_b ] &String (reference to b)   │  
    // └─────────────────────────────────┘  
  
    let a = String::from("alpha");    // SP→ a  
    let b = String::from("beta");     // SP→ a, b  
  
    let r_a = &a;                      // SP→ a, b, r_a  
    let r_b = &b;                      // SP→ a, b, r_a, r_b  
  
    println!("{r_a} {r_b}");           // last use of r_a AND r_b  
    // — r_a and r_b are now "dead" —  
    // SP→ a, b, [r_a dead], [r_b dead]  
  
    println!("{a}");                   // last use of a  
    // — a is now "dead" —  
    // SP→ [a dead], b, [r_a dead], [r_b dead]  
  
    println!("{b}");                   // last use of b  
    // — b is now "dead" —  
  
    // Implicit drops (reverse order):  
    // drop(b) → free beta heap, pop b stack slot  
    // drop(a) → free alpha heap, pop a stack slot  
    // (r_a, r_b already dead, no drop needed)  
}
```


Summary

Concept	What happens
Registering a pointer	Rust places the reference on the stack; the borrow checker records its liveness
Frame effect	Same mechanical stack as C; compiler tracks which slots are "alive" at each program point
Drop schedule	Compiled in at compile time — deterministic, no GC, no RC
Scope limiter	Borrow checker (NLL) — ends borrows at last use, not at end of scope
Frame adjustment	Dead slots are invisible to the borrow checker; LLVM may reuse them
Design principle	Small scopes → short borrows → compiler is happy → your code compiles
Flow coordination	Let the type system encode the scope constraints; use blocks to bound borrows; prefer owned at boundaries